

Tu *parte web* y cómo enlaza con el CRM

Lo justo y necesario para construir el Hub en la web: stack, datos que muestras, módulos UI por rol, y el contrato exacto con GoHighLevel (lectura, escritura y webhooks).

DIRIGIDO A

Javi · Z7 Media

OBJETIVO

Función · Web + Enlace CRM

VERSIÓN

v1.0 · Mayo 2026

Qué *construyes*

TU ALCANCE · WEB Y ENLACE CON GHL

01

Tu función en una frase

Construir y mantener el **frontend SPA + backend ligero** del Hub interno de Closers / Colaboradores, y orquestar el enlace bidireccional con el CRM (GoHighLevel) que mantiene Claudia.

Lo que vive en tu lado (Web)

SÍ es tu responsabilidad

- **SPA Hub**: login, routing, render por rol, componentes UI.
- **Backend ligero**: proxy autenticado a GHL + receptor de webhooks.
- **BBDD propia** del Hub (users, snapshots CVR, liquidaciones, audit log).
- **Auth**: login + sesión + control de roles.
- **Hosting** y disponibilidad del Hub.
- **Performance** y cache de llamadas GHL.

NO es tu responsabilidad

- Configuración del CRM GHL (Claudia).
- Decisión de fórmulas / % comisión (Adolfo).
- Asignación de leads y aprobación de liquidaciones (Ana / Adolfo).
- Comunicaciones SMS/email al cliente (workflows GHL, Claudia).
- Captura inicial del lead en la web pública (vive en GHL).

El contrato con GHL en 4 puntos

1. **Lees** de GHL: leads, contactos, oportunidades, pipeline, usuarios.
2. **Escribes** a GHL: cambios de stage, notas, llamadas, tags (acciones del closer).
3. **Recibes** webhooks GHL en tu endpoint `/api/hub/ghl/webhook` para mantenerte fresco.
4. **Calculas** en tu lado: CVR, liquidaciones, agregados (lo que GHL no calcula).

Regla de oro. GHL es source-of-truth de leads y stages. El Hub es source-of-truth de usuarios internos, CVR y liquidaciones. Si hay conflicto en un campo de lead, gana GHL. Si hay conflicto en CVR o comisión, gana el Hub.

Arquitectura *SPA*

STACK RECOMENDADO · PATRONES · DIAGRAMA

02

Stack

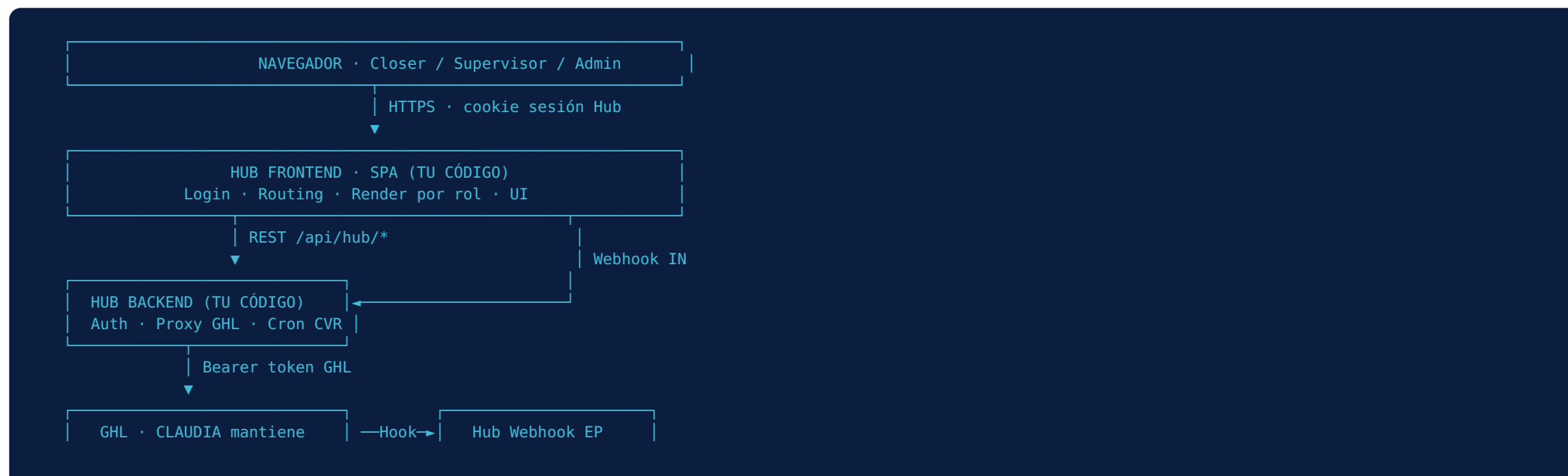
Frontend

- HTML + CSS + JS (ES2022) o React si prefieres.
- Single-file SPA con hash routing.
- Sesión en cookie HTTP-only + JWT en backend.
- Fetch API contra tu backend (nunca contra GHL directo).

Backend

- Node.js + Express (o similar).
- Endpoints `/api/hub/*`.
- Proxy autenticado a GHL con bearer token.
- Receptor webhook con verificación HMAC.
- PostgreSQL para users / snapshots / liquidaciones.

Diagrama



Decisiones clave

DECISIÓN	RECOMENDACIÓN
Token GHL	Sólo en backend. Frontend nunca lo toca.
Cache GHL	TTL 5 min en backend para queries de listado. Sin cache para detalle.
Rate limit GHL	Middleware con cuota 100 req/min. Si se satura → 429 al frontend.
Source-of-truth	Lead = GHL. Usuario / CVR / Liquidación = Hub.
Mobile	Responsive web, no app nativa por ahora.

Entidades *de datos*

ESQUEMAS JSON QUE MANEJAS

03

User (vive en tu BBDD)

```
{
  "id": "u_tamara",
  "email": "tamara@health4spain.com",
  "passwordHash": "sha256:...",
  "rol": "closer", // admin | supervisor | tecnico | closer
  "supervisor_id": "u_ana",
  "ghl_user_id": "abc123", // mapeo con GHL
  "productos_asignados": ["asisa", "lbs"],
  "activo": true
}
```

Lead (espejo de GHL · lectura preferente)

```
{
  "id": "ghl_lead_456",
  "ghl_opportunity_id": "opp_789",
  "nombre": "Juan Pérez",
  "telefono": "+34600000000",
  "producto": "asisa", // custom field GHL
  "seguro": "decesos", // custom field GHL
  "stage": "Cualificada", // de los 9 stages
  "assigned_to": "u_tamara",
  "prima_estimada": 850,
  "tags": ["alta-prioridad"]
}
```

Snapshot CVR (lo calcula tu cron, vive en tu BBDD)

```
{
  "user_id": "u_tamara",
  "fecha": "2026-05-15",
}
```

```

"leads_recibidos_30d": 87,
"leads_cerrados_30d": 9,
"cvr": 10.34,
"nivel": "objetivo",
"bonus_pct": 0.25
}
// La FÓRMULA de CVR la define Adolfo. Tú la implementas con valores configurables en config.

```

Liquidación (vive en tu BBDD)

```

{
  "id": "liq_2026_05_u_tamara",
  "user_id": "u_tamara",
  "periodo": "2026-05",
  "regimen": "n+1", // viene del producto del lead
  "estado": "pendiente", // pendiente | aprobada | liquidada
  "items": [
    { "lead_id": "456", "comision_neta": 63.75 },
    { "lead_id": "457", "comision_neta": 61.20 }
  ],
  "total_neto_closer": 124.95
}
// Los % por compañía vienen de tabla config_comisiones EDITABLE por Admin (no hardcoded).

```

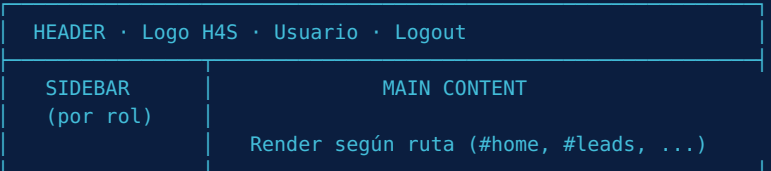
⚠ **Lo que NO modelas tú.** No diseñes tablas propias para "contactos", "tareas", "comunicaciones" — todo eso vive en GHL. Tu BBDD sólo guarda: users, snapshots_cvr, liquidaciones, audit_log y processed_events (idempotencia webhooks).

Módulos *UI* · por rol

LAS PANTALLAS QUE TIENES QUE CONSTRUIR

04

Layout general



Pantallas por rol

PANTALLA	CLOSER	SUPERVISOR	ADMIN	TÉCNICO
Home	KPIs propios	KPIs equipo	KPIs global	KPIs sistema
Mis Leads	Sí	vista equipo	vista global	—
Mi CVR	Sí	Sí	Sí	—
Mis Comisiones	Sí	vista equipo	vista global	—
Mi Equipo	—	Sí	Sí	—
Productos (ASISA / LBS / Partners / Com / Energía)	según asignación	todos	todos	todos
Configuración	—	—	Sí	—
Integraciones / Logs	—	—	Sí	Sí

Mis Leads (la pantalla más usada)

- Tabla paginada con leads del closer logeado (filtros por stage, producto, antigüedad).

- Botón "**Llamar**" → tel: link + `POST /api/hub/leads/:id/call` al backend → registra actividad en GHL.
- Botón "**Cambiar stage**" → modal → `PATCH /api/hub/leads/:id/stage` → propaga a GHL.
- Botón "**Marcar cerrada**" → modal con prima + fecha firma + ref compañía → `POST /api/hub/leads/:id/close` → propaga a GHL + genera item liquidación.

Punto crítico. Cualquier escritura desde Hub a GHL debe ser **síncrona** con feedback al usuario. Si GHL falla, mostrar error en UI y reintentar en background (no perder el cambio).

Enlace con *GHL*

ENDPOINTS QUE TU BACKEND LLAMA (LECTURA)

05a

Llamadas que tu backend hace a GHL · LECTURA

ENDPOINT GHL	MÉTODO	PARA QUÉ LO USAS
/v2/locations/{loc}/opportunities	GET	Lista de leads asignados a un user_id (Mis Leads).
/v2/locations/{loc}/opportunities/{id}	GET	Detalle de un lead (al abrirlo).
/v2/locations/{loc}/contacts/{id}	GET	Datos del contacto + histórico.
/v2/locations/{loc}/pipelines	GET	Pipeline maestro y sus 9 stages (cache 1 día).
/v2/locations/{loc}/users	GET	Lista de users GHL para mapear con users Hub.
/v2/locations/{loc}/custom-fields	GET	Esquema de custom fields (cache 1 día).

Ejemplo · GET opportunities

```
GET /v2/locations/loc_h4s/opportunities?assignedTo=ghl_user_tamara&pipelineId=pl_h4s_maestro
Authorization: Bearer {GHL_TOKEN}
```

```
200 OK
{
  "opportunities": [
    {
      "id": "opp_456",
      "contactId": "cnt_789",
      "name": "Juan Pérez",
      "pipelineStageId": "stage_cualif",
      "status": "open",
      "monetaryValue": 850,
      "assignedTo": "ghl_user_tamara",
      "customFields": {
        "producto": "asisa",
        "seguro": "decesos",
        "prima_estimada": 850
      }
    }
  ]
}
```

```
    },  
    "tags": ["alta-prioridad"],  
    "createdAt": "2026-05-10T09:32:00Z"  
  }  
],  
"meta": { "total": 42 }  
}
```

Lo que demandas a Claudia para empezar

1. **Bearer token GHL** con scopes adecuados.
2. **location_id** de la sub-account H4S.
3. **pipeline_id** + **stage_id** de cada uno de los 9 stages.
4. JSON con **custom_field IDs** y sus nombres técnicos.
5. CSV con **mapping ghl_user_id ↔ email closer**.
6. Sub-account de **staging** separada para tests.
7. **Webhook secret** para verificar firmas HMAC.

Enlace con *GHL*

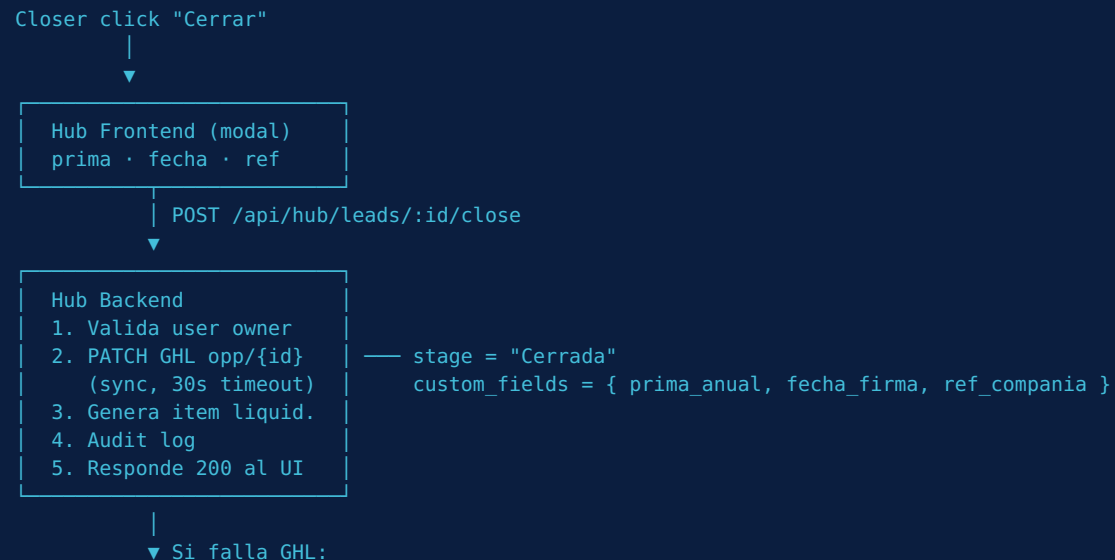
ENDPOINTS QUE TU BACKEND LLAMA (ESCRITURA)

05b

Llamadas que tu backend hace a GHL · ESCRITURA

ENDPOINT GHL	MÉTODO	CUÁNDO LO LLAMAS
/v2/locations/{loc}/opportunities/{id}	PATCH	Cambio de stage / custom fields al cerrar venta.
/v2/locations/{loc}/contacts/{id}/notes	POST	Añadir nota al contacto desde el Hub.
/v2/locations/{loc}/contacts/{id}/tags	POST	Añadir tag (ej. alta-prioridad).
/v2/locations/{loc}/calls	POST	Registrar resultado de una llamada manual.
/v2/locations/{loc}/workflows/{wf}/trigger	POST	Disparar workflow (ej. rescate).

Flujo crítico · cierre desde Hub propaga a GHL



Retry exponencial (1s, 5s, 30s) → queue persistente
Frontend muestra "Cierre registrado, sincronizando..."

Manejo de errores GHL

ERROR	ESTRATEGIA
401 / 403	Token expirado o sin permiso → escala a Claudia. Muestra al user "Permiso denegado".
404	Lead no existe en GHL → resync con webhook reciente. Muestra "Lead no encontrado, recarga".
409	Conflict (lead cambió tras tu fetch) → resync y pide al user confirmar de nuevo.
429	Rate limit → cola interna con retry exponencial.
5xx	GHL caído → retry 3 veces. Si sigue → queue persistente + alerta.

Webhooks *de GHL*

LO QUE RECIBES Y CÓMO LO PROCESAS

06

Endpoint receptor

```
POST /api/hub/ghl/webhook
Headers:
  X-GHL-Signature: sha256=...    ← verificala SIEMPRE
  Content-Type: application/json

Body:
{
  "event_id": "evt_abc",          ← idempotencia
  "event_type": "OpportunityStageChange",
  "data": { ... }
}
```

Eventos que Claudia te envía

EVENTO	QUÉ HACES EN EL HUB AL RECIBIRLO
ContactCreate	Inserta lead en BBDD Hub si no existe.
OpportunityCreate	Asocia oportunidad a contacto. Refresca vista del closer asignado.
OpportunityStageChange	Actualiza stage en Hub. Si nuevo stage = "Cerrada", dispara recálculo CVR.
UserAssigned	Cambia <code>assigned_to</code> en Hub. Notifica al closer (badge).
TagAdded / TagRemoved	Actualiza tags del lead.
NoteAdded	Refleja en timeline del lead en Hub.
TaskCreated / TaskCompleted	Sincroniza tareas en timeline.
InboundMessage	Notifica al closer en Mis Leads.

Idempotencia y firma · obligatorias

```
// Middleware
function handleWebhook(req, res) {
  // 1. Verificar firma
  const expected = HMAC_SHA256(req.body_raw, WEBHOOK_SECRET);
  if (req.headers['x-ghl-signature'] !== `sha256=${expected}`) {
    return res.status(401).end();
  }
  // 2. Idempotencia
  if (await db.exists('processed_events', req.body.event_id)) {
    return res.status(200).json({ skipped: 'duplicate' });
  }
  // 3. Procesar evento
  await processEvent(req.body);
  await db.insert('processed_events', { event_id: req.body.event_id });
  res.status(200).json({ ok: true });
}
```

Sync nocturno de seguridad. Por si un webhook se perdió: cron 04:00 que pide a GHL `GET /opportunities?updated_since={last_sync}` y reconcilia con tu BBDD. Detecta y corrige drift.

Auth *y seguridad*

LO MÍNIMO IMPRESCINDIBLE PARA PRODUCCIÓN

07

Login y sesión

- Login email + password. Hash `sha256(password + salt)` en BBDD.
- Sesión JWT (HS256) con expiración 8h.
- Cookie HTTP-only · Secure · SameSite=Strict.
- Logout = revoke JWT (blacklist Redis con TTL 8h).
- 2FA TOTP **obligatorio para Admin y Supervisor** antes del go-live.

Control de acceso (RBAC)

Middleware central que valida rol en cada endpoint sensible:

```
app.post('/api/hub/users', requireRole('admin'), handler);
app.patch('/api/hub/leads/:id/stage', requireRole('admin','supervisor','closer'), handler);
app.post('/api/hub/liquidaciones/:id/approve', requireRole('admin','supervisor'), handler);
app.get('/api/hub/audit', requireRole('admin','tecnico'), handler);
```

Tokens y secretos

- Bearer token GHL: **sólo en .env del backend**. NUNCA en frontend.
- Webhook secret: rotación cada 6 meses con ventana 1h de doble validación.
- JWT secret: rotación cada 12 meses (revoca todas las sesiones activas).
- Gestor de passwords (1Password / Bitwarden) para compartir con Claudia.

Audit log mínimo

Tabla `audit_log` en tu BBDD con cada acción sensible:

```
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
```

```
ts TIMESTAMPTZ DEFAULT NOW(),
user_id VARCHAR(32),
user_role VARCHAR(16),
ip_address INET,
action VARCHAR(64),          -- 'login', 'lead_close', 'user_create', ...
resource_type VARCHAR(32),
resource_id VARCHAR(64),
metadata JSONB,
result VARCHAR(16)
);
```

⚠ **GDPR.** El Hub guarda PII espejada de GHL. Endpoint `/api/hub/admin/gdpr/forget` elimina lead en Hub y propaga a GHL. Backups encriptados (AES-256) con retención 90 días. TLS 1.3 obligatorio.

Roadmap *de sprints*

8 SPRINTS · 16 SEMANAS · MVP EN S4

08

Plan quincenal

SPRINT	SEMANAS	HITO	ENTREGA JAVI	DEPENDE DE CLAUDIA
S1	1-2	Esqueleto SPA + Auth	SPA login, sesión, 4 roles render placeholder	CSV users mapping
S2	3-4	Conexión GHL lectura	Proxy backend a GHL, vista Mis Leads	Token GHL · pipeline_id · custom fields IDs
S3	5-6	Webhooks entrantes	Endpoint webhook con firma + idempotencia	Webhook secret · workflows configurados
S4	7-8	MVP Closer	Llamar · cambiar stage · cerrar	Pipeline stages estables · tags maestros
S5	9-10	Motor CVR	Cron diario, snapshots, gauge UI, bonus aplicado	Histórico inicial leads para warm-up
S6	11-12	Liquidaciones	UI consulta closer + aprobación supervisor	Tabla config_comisiones por compañía
S7	13-14	Módulos por producto	UI ASISA, LBS, Partners, Com, Energía + calculadora	Material de venta · % por producto
S8	15-16	Hardening + 2FA + Audit	2FA, dashboard audit, polish UX, doc	Validación final QA conjunta

Lo que demandas en cada sprint

- **S1:** CSV con ghl_user_id ↔ email closer ↔ rol Hub.
- **S2:** Token GHL + location_id + pipeline_id + custom fields IDs + sandbox.
- **S3:** Webhook secret + lista de eventos GHL que vais a emitir.
- **S4:** Tabla de tags maestros congelada + stages del pipeline congelados.
- **S5:** Histórico de leads cerrados últimos 90 días para validar fórmula CVR.
- **S6:** Tabla `config_comisiones` con % por producto / compañía actualizado.

Riesgos a vigilar

Técnicos

- GHL rate limits → cache + paginación obligatorios.
- Webhooks perdidos → sync nocturno reconcilia.
- Drift Hub vs GHL → audit semanal automático.

De proyecto

- Cambios de fórmula CVR / % comisión → mantén tablas editables.
- Nuevos productos → arquitectura ya modular.
- Dependencia con Claudia → frozen specs en S2 y S4.

Próximos pasos. (1) Confirma stack final y crea repo. (2) Pide a Claudia el token GHL y custom fields. (3) Kick-off técnico Javi + Claudia + Adolfo para alinear stages y custom fields. (4) Sandbox GHL operativo antes de S2.